

LifeOS

A Student Campus Life Manager

CSE1021 - Introduction to Problem Solving and Programming

Fall Semester 2026-27 | Python 3 | Terminal application

1

Life Map
& Profile

2

Action Board
Tasks

3

Timetable
& Rooms

4

Expense
Tracker

SUBMITTED BY	Sarthak Srivastava
REGISTRATION NO.	26BCE11481
PROGRAMME	B.Tech Computer Science Engineering (1st Year)
FACULTY	Ramraj Dangi
GITHUB	github.com/Sarthak26Srivastava/LifeOs

Contents

2. Introduction

3

3. Problem Statement

3

4. Functional Requirements

4

5. Non-Functional Requirements

5

6. System Architecture

5

7. Design Diagrams

7

8. Design Decisions and Rationale

11

9. Implementation Details

11

10. Screenshots / Results

13

11. Testing Approach

15

12. Challenges Faced

16

13. Learnings and Key Takeaways

16

14. Future Enhancements

17

15. References

18

A: Appendix: Evaluation Rubric Alignment

18

Project at a glance

4

functional modules

7

Python files

47

automated checks

21

manual test cases

The idea in one line

One menu, one terminal, one Python program that keeps a first-year student's tasks, classes, study rooms, clubs and pocket money in one place - with Study and Health scores and milestones that make progress visible.

2 Introduction

LifeOS is a menu-driven, terminal-based student life management system written in Python 3 for the CSE1021 course. It brings the everyday activities of a first-year campus student into one program: a personal profile and diary, task and deadline management, a class timetable, study-room booking and pocket-money tracking. Completing tasks raises a Study score and unlocks milestone messages; spending money raises a Health value. Progress in different areas of student life is therefore visible in one place.

The project applies the concepts of the course (variables, input/output, operators, conditionals, loops, lists, dictionaries, strings, functions and modules) to a real problem instead of isolated exercises. It runs entirely in the terminal, needs no internet connection, database, GUI or external library, and works on any system with Python 3.

Objectives

- Create a simple student life management system that solves a practical problem.
- Apply the Python concepts of CSE1021 to a real-world scenario.
- Divide the application into independent functional modules.
- Provide clear input and output through a menu-driven terminal interface.
- Validate input and handle common mistakes without crashing.
- Share student information (name, Health, Study, Money) across modules.
- Test the program with an automated test file and a manual test plan.
- Leave a foundation that can later be extended with persistent storage and new features.

3 Problem Statement

College students manage many small things at once: assignments with deadlines, class timetables, study rooms that may already be occupied, clubs to join, personal events and a limited pocket-money budget. These are usually handled separately or simply remembered, so it is easy to forget a deadline, try to book an occupied room or lose track of spending. An ordinary to-do list covers only tasks and ignores the rest of student life.

LifeOS gives one lightweight terminal application where these activities are managed from a single menu. It validates input and gives clear feedback to prevent invalid operations: duplicate room bookings, overspending, invalid priorities, empty task names, wrongly sized deadlines and non-numeric entries where a number is expected.

Scope and target users

LifeOS is designed for **one student per program session**, mainly first-year students living on campus. Multi-user login, a database, permanent storage, notifications and a graphical interface are outside the current version and are listed under Future Enhancements.

4 Functional Requirements

LifeOS has four major functional modules plus a main menu that connects them. The table also shows where each requirement is implemented and how it is verified, so every requirement can be traced to code and to a test.

ID	Requirement (the system shall...)	Implemented in	Verified by
FR1 Main menu	Ask for the student's name at start; show the menu until the student quits; reject invalid choices with a message; ask for confirmation before quitting.	main.py	Manual tests 1, 20, 21
FR2 Profile, diary, clubs	Display name, Health, Study and Money; display the journey diary; add a diary event (empty events rejected); join the Coding Club or Sports Club; prevent joining a club twice; reject unknown clubs.	life_map.py student_stats.py	Automated: Life Map (7 checks) Manual 11
FR3 Task management	Add a task with name, priority (high / medium / low) and deadline; view tasks; mark a task completed only once; delete a task; show a total / done / pending summary with percentage; list pending tasks grouped by priority; raise Study on completion and write milestones (first task, Study 50); reject missing or non-numeric task numbers.	action_board.py	Automated: Action Board (18) + Delete task (6) Manual 2-10, 19
FR4 Timetable and study rooms	Add subjects to the timetable (empty names rejected); view the timetable and say when it is empty; offer three study rooms; book an available room; prevent double booking; reject invalid rooms.	timetable_rooms.py	Automated: Timetable and Rooms (7) Manual 12, 13, 17, 18
FR5 Expense management	Display the balance; add pocket money; record spending; refuse spending above the balance; reject zero, negative and non-numeric amounts; raise Health by 10 (maximum 100) when money is spent; show the updated balance.	expense_tracker.py	Automated: Expense Tracker (9) Manual 14, 15, 16

Input / output structure

Inputs	Outputs
Student name, menu choices, task name / priority / deadline / number, diary events, club names, subject names, room selection, money amounts, quit confirmation.	Profile and diary, club status, task list and summary, Study score and milestones, timetable, room-booking status, money balance, transaction results, validation and error messages.

5 Non-Functional Requirements

ID	Quality	How LifeOS meets it
NFR1	Usability	Simple numbered menu, clear prompts; allowed values (for example high / medium / low) are shown wherever input is needed.
NFR2	Reliability	Invalid operations are blocked: duplicate club membership, duplicate room booking, completing a task twice, overspending and non-numeric input.
NFR3	Maintainability	One file per responsibility; shared values kept in <code>student_stats.py</code> , so a feature can change without rewriting the application.
NFR4	Error handling	Important inputs are checked before use (for example <code>isdigit()</code> before <code>int()</code>), giving a short message instead of a crash.
NFR5	Resource efficiency	Only built-in Python features and in-memory data; every action responds instantly.
NFR6	Portability	Runs on Windows, macOS and Linux with Python 3 and standard terminal input and output.
NFR7	Testability	<code>input()</code> can be replaced by prepared answers, so every function can be tested without typing.

6 System Architecture

LifeOS uses a simple hub-and-spoke modular architecture. **main.py** is the hub: it shows the menu and calls one of four feature modules. Each feature module reads and updates the shared values in **student_stats.py**, which acts as a small shared data store. The only direct link between feature modules is that **action_board.py** imports **life_map.py**, so a completed task can write a milestone into the journey diary.

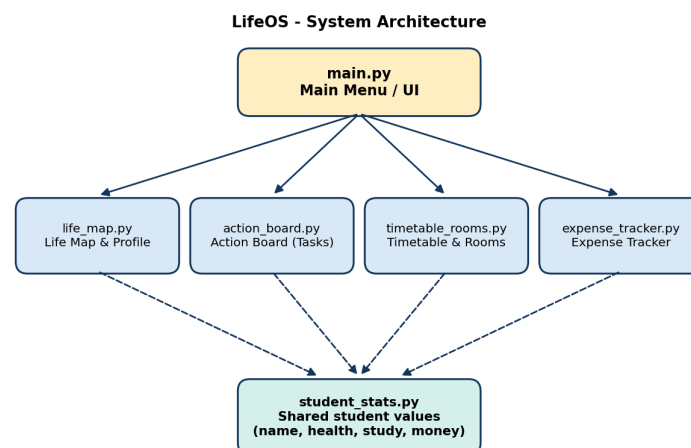


Figure 1: System architecture of LifeOS

Project structure

```
LifeOs/  
|-- main.py           menu and program loop  
|-- student_stats.py  shared student values  
|-- life_map.py       profile, diary, clubs  
|-- action_board.py   tasks, priorities, milestones  
|-- timetable_rooms.py timetable and study rooms  
|-- expense_tracker.py pocket money and spending  
|-- test_lifeos.py    automated checks (47)  
|-- README.md, statement.md  
`-- screenshots/      menu, tasks, rooms, expense
```

7 Design Diagrams

7.1 Use Case Diagram

The only actor is the Student. Every use case is reached through the main menu.

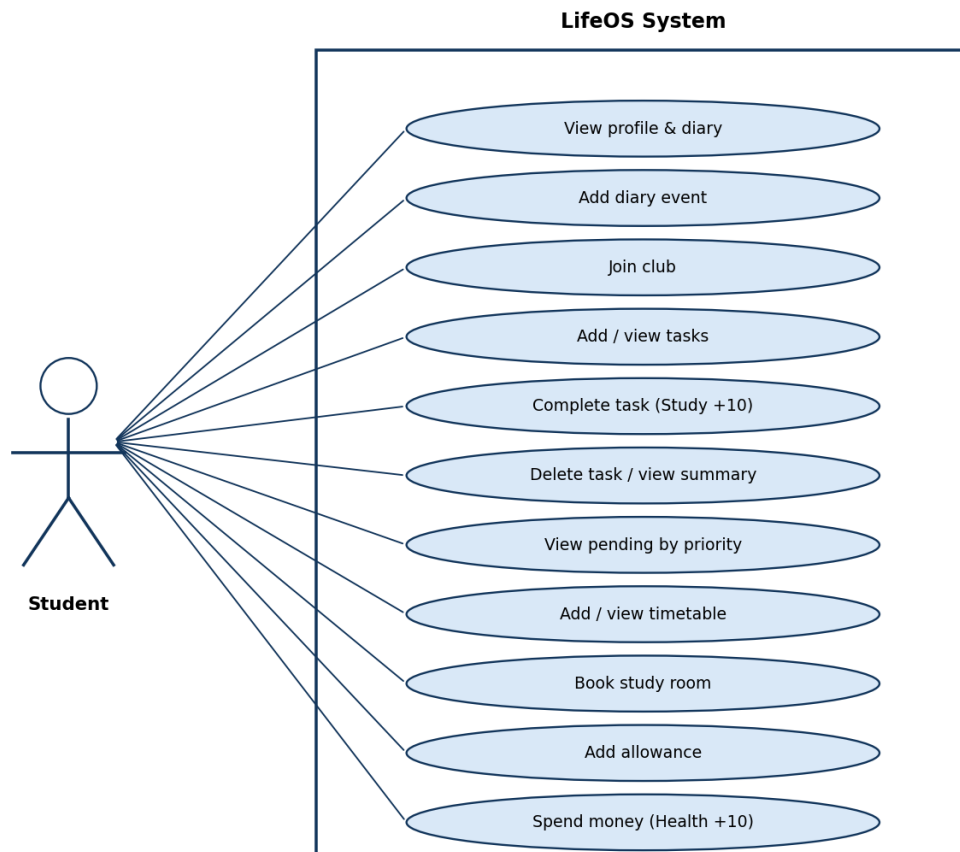


Figure 2: Use case diagram

7.2 Workflow Diagram

The menu repeats until the student confirms quitting. An invalid choice and a declined quit both return to the menu.

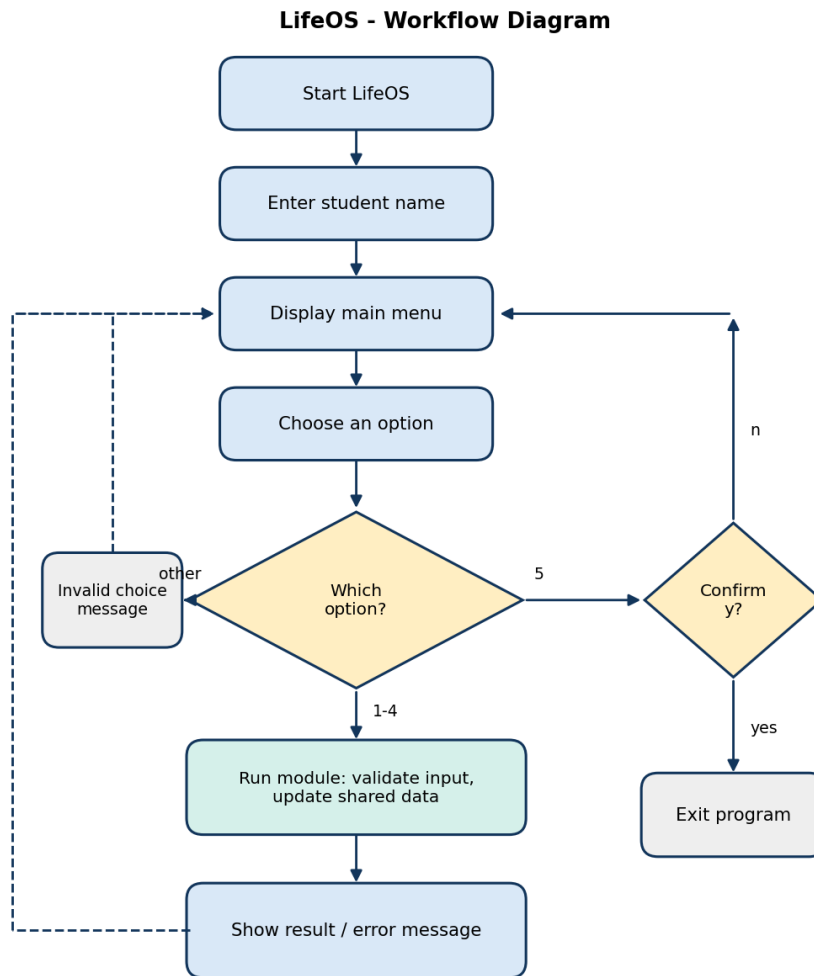


Figure 3: Workflow of LifeOS

7.3 Sequence Diagram

The most important interaction in the system is completing a task, because it touches three modules.

LifeOS - Sequence Diagram: Completing a Task

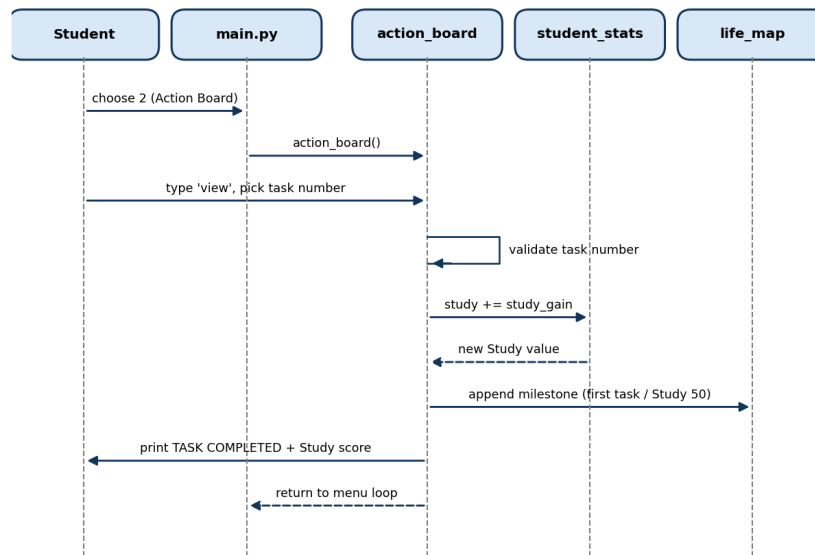


Figure 4: Sequence diagram - completing a task

7.4 Class / Component Diagram

LifeOS uses functions and module-level data instead of classes, so its structure is shown as a component diagram: each box is a Python module with its data and functions.

LifeOS - Class / Component Diagram

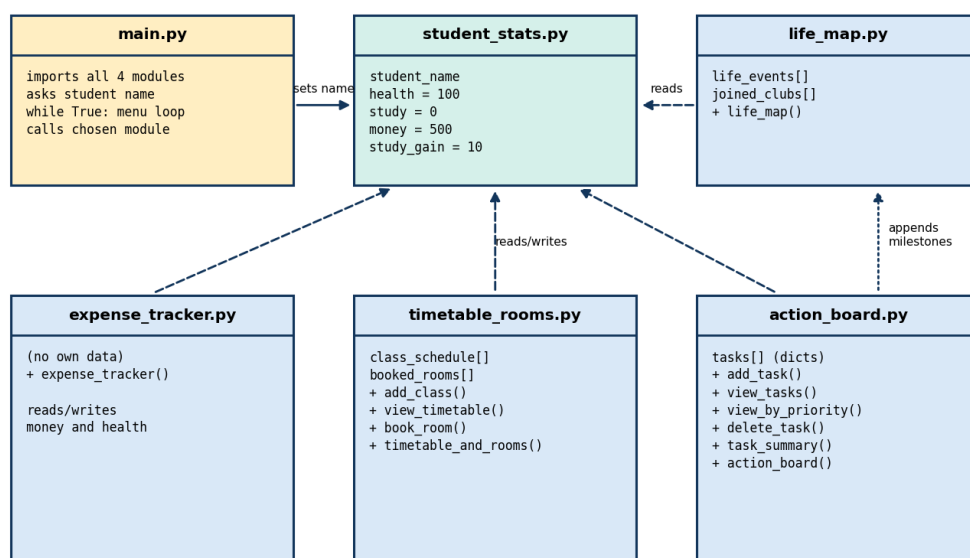


Figure 5: Component diagram

7.5 ER Diagram / Storage Design

Not applicable. LifeOS keeps all data in memory and uses no database or files, so there is no ER diagram or schema. The in-memory structures are: `tasks` (a list of dictionaries with name, priority, deadline and status), `life_events`, `joined_clubs`, `class_schedule` and `booked_rooms` (lists of strings) and the shared values in `student_stats.py`.

8

Design Decisions and Rationale

Decision	Why
A separate module for each feature	Small readable files, features can change independently, and it matches the course idea of breaking a problem into smaller parts.
One shared file, student_stats.py	Health, Study, Money and name are needed by several modules; one shared place keeps them consistent without passing many values around.
Access values as student_stats.money (module attribute)	Importing a number directly copies it, so an update in one module would not be seen in another. Attribute access keeps one live value.
Tasks stored as dictionaries in a list	Each task has several fields; a dictionary keeps them together and is easy to print, count and delete.
Check input before converting	Testing isdigit() before int() prevents ValueError crashes without needing advanced exception handling.
Fixed lists of valid options	Priorities, clubs and rooms are validated with a simple 'in list' check, keeping behaviour predictable.
Study score, Health and milestones	Give visible feedback so that using the tool feels rewarding.
In-memory storage only	Keeps the program dependency-free and focused on core Python; file storage is planned as an enhancement.
A test file that replaces input()	Every function can be tested repeatedly without typing, and mistakes are caught early.

9

Implementation Details

Python concepts used: variables, data types, input/output, operators, if / elif / else, while and for loops, lists, dictionaries, strings, functions, modules and conditional validation.

File	What it does and how
main.py	Imports the four feature functions, stores the entered name in student_stats and runs a while True loop that prints the menu and calls the chosen function. Choice 5 asks for confirmation (y or Y exits, anything else returns to the menu); other values print an invalid-choice message.
student_stats.py	Holds student_name, health (100), study (0), money (500) and study_gain (10 points per completed task).
life_map.py	Prints the profile, joined clubs and journey list (starting with 'Joined College'), then asks for 'add' (append a non-empty event) or 'club' (Coding Club / Sports Club; unknown or repeated clubs are rejected). Joining a club is also logged as an event.

File	What it does and how
action_board.py	Handles add / view / priority / delete / summary. add_task checks for an empty name, a priority in [high, medium, low] and a deadline of 10 characters. view_tasks lists the tasks and lets the student mark one done: it checks isdigit(), range and already-Done, adds study_gain to Study and writes milestones. view_by_priority prints pending tasks for high, medium and low. task_summary counts tasks and prints the whole-number percentage completed.
timetable_rooms.py	add_class appends a non-empty subject; view_timetable prints numbered subjects or a 'no classes' message; book_room accepts only study room 1, 2 or 3 and refuses a room that is already in booked_rooms.
expense_tracker.py	Shows the balance, accepts 'spend' or 'add', reads the amount as text, checks isdigit(), converts to int and rejects 0. Spending above the balance is refused; valid spending lowers money and raises Health by 10, capped at 100.
test_lifeos.py	Replaces builtins.input with a function that returns prepared answers, resets all data before each group, calls each function and prints PASS or FAIL for 47 checks.

Key code: completing a task

```

before = student_stats.study
tasks[task_index]["status"] = "Done"
student_stats.study += student_stats.study_gain
print("--- TASK COMPLETED --- Study is now", student_stats.study)
if before == 0:
    print("Milestone: first task completed!")
    life_map.life_events.append("Completed first task")
if before < 50 and student_stats.study >= 50:
    print("Milestone: Study reached 50!")
    life_map.life_events.append("Study reached 50")

```

Key code: safe numeric input

```

amount_text = input("Amount: Rs ")
if not amount_text.isdigit():
    print("Please type the amount as a whole number.")
    return
amount = int(amount_text) # only digits reach this line

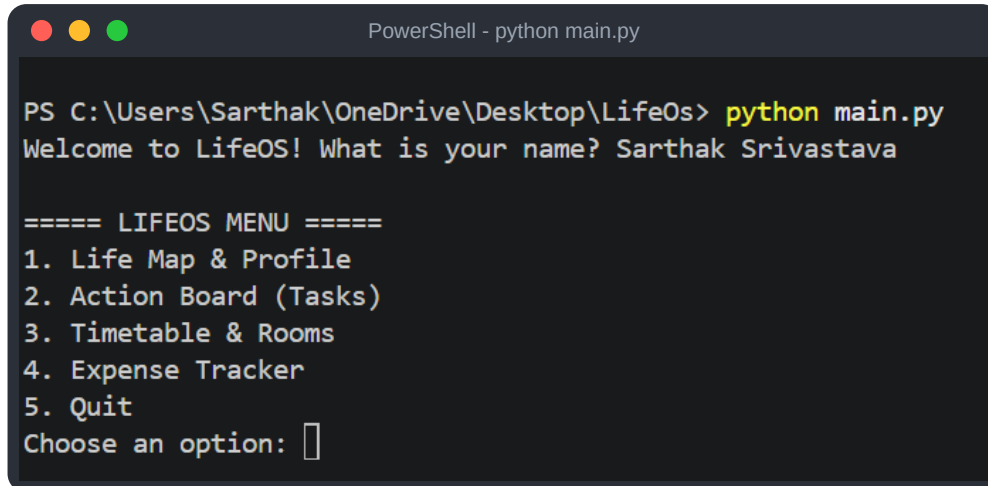
```

Version control

The project is maintained with Git and hosted on GitHub at github.com/Sarthak26Srivastava/LifeOs. The repository holds the source code, test file, README.md, statement.md and the screenshots folder.

10 Screenshots / Results

The screens below were captured from the running program in the VS Code terminal on Windows.

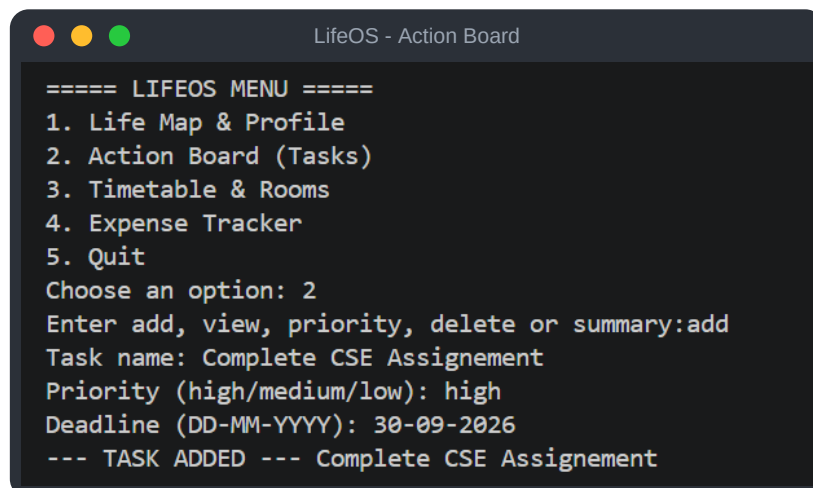


```
PowerShell - python main.py

PS C:\Users\Sarthak\OneDrive\Desktop\LifeOs> python main.py
Welcome to LifeOS! What is your name? Sarthak Srivastava

===== LIFEOS MENU =====
1. Life Map & Profile
2. Action Board (Tasks)
3. Timetable & Rooms
4. Expense Tracker
5. Quit
Choose an option: 
```

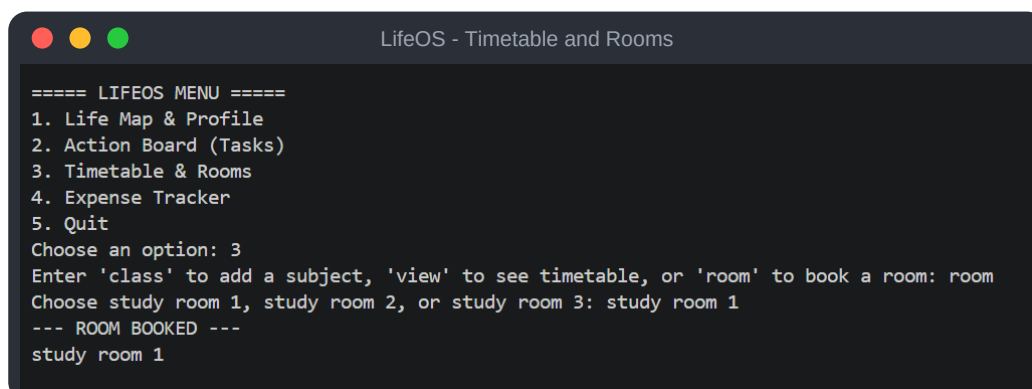
Figure 6: Program start and main menu



```
LifeOS - Action Board

===== LIFEOS MENU =====
1. Life Map & Profile
2. Action Board (Tasks)
3. Timetable & Rooms
4. Expense Tracker
5. Quit
Choose an option: 2
Enter add, view, priority, delete or summary: add
Task name: Complete CSE Assignment
Priority (high/medium/low): high
Deadline (DD-MM-YYYY): 30-09-2026
--- TASK ADDED --- Complete CSE Assignment
```

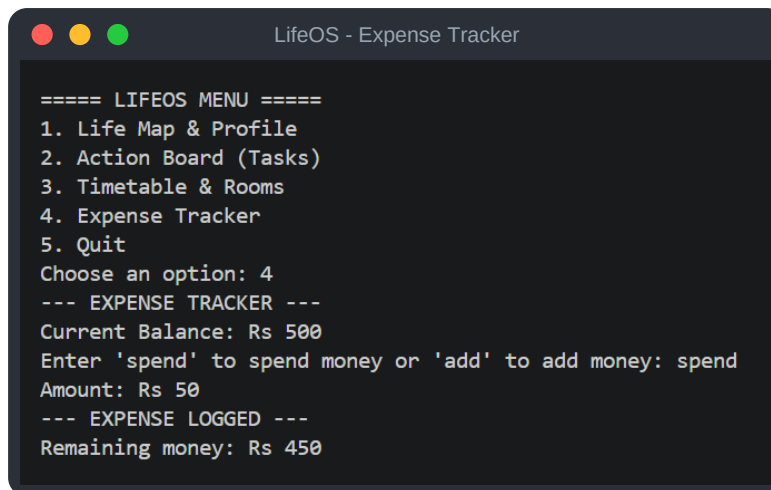
Figure 7: Adding a task in the Action Board



```
LifeOS - Timetable and Rooms

===== LIFEOS MENU =====
1. Life Map & Profile
2. Action Board (Tasks)
3. Timetable & Rooms
4. Expense Tracker
5. Quit
Choose an option: 3
Enter 'class' to add a subject, 'view' to see timetable, or 'room' to book a room: room
Choose study room 1, study room 2, or study room 3: study room 1
--- ROOM BOOKED ---
study room 1
```

Figure 8: Booking a study room

A screenshot of a terminal window titled "LifeOS - Expense Tracker". The window has a dark background with light-colored text. The text shows a menu with five options: 1. Life Map & Profile, 2. Action Board (Tasks), 3. Timetable & Rooms, 4. Expense Tracker, and 5. Quit. Option 4 is selected. Below the menu, it says "Choose an option: 4". Then, it shows "--- EXPENSE TRACKER ---", "Current Balance: Rs 500", and a prompt "Enter 'spend' to spend money or 'add' to add money: spend". The user has entered "spend", and it shows "Amount: Rs 50". Then, it says "--- EXPENSE LOGGED ---" and "Remaining money: Rs 450".

```
==== LIFEOS MENU ====
1. Life Map & Profile
2. Action Board (Tasks)
3. Timetable & Rooms
4. Expense Tracker
5. Quit
Choose an option: 4
--- EXPENSE TRACKER ---
Current Balance: Rs 500
Enter 'spend' to spend money or 'add' to add money: spend
Amount: Rs 50
--- EXPENSE LOGGED ---
Remaining money: Rs 450
```

Figure 9: Spending Rs 50 (balance 500 to 450)

11 Testing Approach

47

automated checks

5

test groups

21

manual cases

Automated testing

`test_lifeos.py` swaps the real `input()` for a fake one that returns prepared answers, calls each function directly and prints PASS or FAIL. Run it with `python test_lifeos.py`; no typing is needed. The 47 checks are: Action Board 18, Delete task 6, Life Map 7, Timetable and Rooms 7, Expense Tracker 9. They cover valid input, invalid input and edge cases such as an empty list, pressing Enter to skip and Health never exceeding 100. Three of the checks are smoke checks that only confirm a screen runs without an error. All 47 checks pass.

Manual testing

#	Test	Expected result
1	Enter 9 at the main menu	Invalid menu choice message
2	Add task with empty name	Empty-name error
3	Enter priority 'urgent'	Invalid priority message
4	Enter deadline 30-9-2026	Invalid date-format message
5	Add and complete a valid task	Task completed, Study increases
6	Complete the same task again	Already-completed message
7	Enter task number 99	Task does not exist message
8	Enter letters for a task number	Please-type-a-number message, no crash
9	Delete a task	Task removed from the list
10	View the task summary	Correct total, done and pending counts
11	Join the same club twice	Duplicate membership message
12	Book the same room twice	Room unavailable message
13	Book room 9	Invalid room message
14	Spend more than the balance	Insufficient money message
15	Enter a negative amount	Invalid amount message
16	Enter letters for an amount	Please-type-the-amount message, no crash
17	View timetable before adding classes	No classes message
18	Add a valid class	Class added successfully
19	Complete five tasks	Study reaches 50 and milestone appears
20	Select Quit and enter n	Returns to main menu
21	Select Quit and enter y	Program exits

12 Challenges Faced

Crashes on wrong input

Typing letters where a number was expected stopped the program. Solved by checking `isdigit()` before converting with `int()`.

Sharing values between modules

Several files needed the same Health, Study and Money values. Keeping them in `student_stats.py` and reading them as `student_stats.money` fixed this.

Repeated and invalid actions

Completing a task twice, booking a room twice or joining a club twice gave wrong results until explicit checks were added.

Milestones firing only once

The Study 50 message must appear only when the score crosses 50, so the old score is saved before the update and compared with the new one.

Testing without typing

Functions call `input()`, which made testing slow. Replacing `input()` with a fake function made tests repeatable.

Time management as a beginner

Learning Python basics, structuring the project and writing documentation alongside regular classes needed careful planning.

13 Learnings and Key Takeaways

- Breaking a problem into smaller modules makes it easier to build, test and fix.
- Designing a menu-driven program with functions and loops.
- Using modules and imports, and how shared data behaves across modules.
- Storing structured data with lists and dictionaries.
- Validating user input, including numeric checks before conversion.
- Writing simple automated tests and a manual test plan.
- Using Git and GitHub for version control and documenting a software project.

14 Future Enhancements

- Save and load data with files or a database so nothing is lost when the program closes.
- Validate deadlines as real calendar dates and add automatic deadline reminders.
- Edit tasks, timetable entries and diary events; allow a booked room to be released.
- Task search, filtering and sorting by deadline; timetable time slots and time-based room booking.
- More clubs and study rooms; wider automated test coverage.
- Graphical user interface, login and multiple student profiles.

Known limitations of the current version

- Data is kept in memory only and is reset when the program closes.
- Choices must be typed exactly as displayed (for example 'study room 1').
- The deadline check confirms only that the text is 10 characters long, not that it is a real date.
- A booked study room cannot be released, and tasks and timetable entries cannot be edited.
- Designed for one student per session; automated tests cover the four feature modules but are not exhaustive.

15 References

- Python Software Foundation, *Python 3 Documentation*, <https://docs.python.org/3/>
- Python Software Foundation, *The Python Tutorial*, <https://docs.python.org/3/tutorial/>
- G. van Rossum, B. Warsaw, N. Coghlan, *PEP 8 - Style Guide for Python Code*, <https://peps.python.org/pep-0008/>
- Git, *Git Documentation*, <https://git-scm.com/doc>
- GitHub, *GitHub Docs*, <https://docs.github.com/>
- VIT Bhopal University, CSE1021 Introduction to Problem Solving and Programming course material, Fall Semester 2026-27.
- VITYarthi, *Build Your Own Project - General Project Instructions and Submission Guidelines*.

A Appendix: Evaluation Rubric Alignment

Rubric component	Weight	Where it is covered
Problem understanding and requirements	10%	Sections 3, 4, 5; statement.md
Design and documentation	20%	Sections 6, 7, 8; README.md; statement.md
Implementation quality	25%	Section 9; seven modular Python files with validation and comments
Innovation, depth and complexity	15%	Study and Health scores, milestones, shared state, priority grouping (Sections 4, 9)
GitHub repository and version control	10%	README.md, statement.md, source, tests, screenshots; Git history
Project report	20%	This document: all 15 required sections